Comparison of Baseline and Improved AI Virtual Mouse Hand-Control Pipelines Using a Point-and-Click Benchmark

Raditya Rafif Pratama Sasmita*
Politeknik Negeri Semarang, Semarang, Indonesia
Corresponding contributor: Raditya Rafif Pratama Sasmita.

ARTICLE INFO
Keywords:
AI virtual mouse
hand gesture recognition
MediaPipe Tasks
MediaPipe Solutions
point-and-click benchmark
cursor control
human-computer interaction

ABSTRACT
Vision-based virtual mouse systems provide a low-cost route for contactless human-computer interaction, but tutorial-style gesture implementations often suffer from unstable click behavior when used in repeated target-acquisition tasks. This study evaluates an AI Virtual Mouse prototype by comparing a baseline hand-control pipeline with an improved pipeline in a controlled point-and-click benchmark. The baseline condition follows tutorial-style gesture semantics: index-only movement and index-plus-middle-finger pinch clicking. The improved condition uses a shared hand-control pipeline with MediaPipe Tasks backend support, adaptive cursor smoothing, stable pinch click detection, click debouncing, optional calibration hooks, and an open-palm pause gesture. To avoid unintended desktop interaction during evaluation, the benchmark uses a simulated cursor inside a Pygame window and never moves the operating-system mouse. Five baseline and five improved hand-input sessions were analyzed. Both conditions achieved 100% target hit rate. However, the improved pipeline reduced average false clicks from 192.8 to 4.2 and reduced average total clicks from 212.8 to 24.2. Average hit-position jitter was similar between conditions, at 10.548 px for baseline and 10.306 px for improved. Mean completion time was not improved: the baseline averaged 3.085 s, while the improved condition averaged 4.963 s because one improved session contained a large outlier. These results support the limited claim that the improved hand-control pipeline greatly reduced unintended clicks while preserving 100% target acquisition success in the controlled benchmark.

1. Introduction

Hand-gesture input has long been studied as a natural interface for human-computer interaction because it allows users to issue commands without a physical pointing device. Recent computer-vision libraries make this approach accessible through webcam-based hand landmark tracking, reducing the need for depth cameras, markers, or wearable hardware. However, a virtual mouse must do more than detect a hand. It must translate noisy landmark positions into stable cursor motion and must distinguish intentional clicks from transient finger configurations.

Many educational AI Virtual Mouse implementations use a compact gesture set: the index finger controls cursor movement, and a pinch-like configuration between the index and middle fingertips triggers a click. This design is easy to demonstrate, but it can produce repeated or unintended click events when the pinch remains active across consecutive frames. Such behavior is especially problematic in point-and-click tasks, where the user must move toward targets and issue discrete selections.

This project evaluates an AI Virtual Mouse experimental prototype that preserves a tutorial-compatible baseline while introducing a more controlled hand-input pipeline. The improved pipeline uses a modern MediaPipe Tasks hand tracker path, adaptive smoothing, stable pinch click logic, debouncing, optional calibration, pause behavior, and shared runtime modules. The evaluation does not attempt to prove that a virtual mouse can replace a physical mouse. Instead, it asks a narrower question: under the same controlled point-and-click benchmark, does the improved hand-control pipeline reduce unintended clicks while preserving target acquisition success?

The main contributions of this paper are as follows:

- A formal comparison between a tutorial-style AI Virtual Mouse baseline and an improved hand-control pipeline.
- A safety-preserving benchmark design that uses a simulated cursor and does not move the operating-system mouse during evaluation.
- A descriptive analysis of five baseline and five improved hand-input benchmark sessions using hit rate, false clicks, total clicks, completion time, and hit-position jitter.
- A conservative interpretation of the observed trade-off between click reliability and completion time.

The structure of this paper follows the experimental style of the reference study [1]: a background section introduces the technology and benchmark context, system design and methodology sections describe the implementation and evaluation, results are reported through tables and cautious interpretation, and the final sections discuss limitations and future work.

2. Related Work / Background

Camera-based hand interaction systems commonly combine hand detection, landmark estimation, gesture classification, and command execution. Earlier virtual mouse systems often used fingertip detection or depth information to map hand motion into cursor movement [2]. Recent work has also explored on-device custom hand gesture recognition for interactive applications [3]. More recent prototypes often use MediaPipe because it provides real-time hand landmark tracking suitable for webcam-based applications [4, 5].

MediaPipe is a framework for building perception pipelines that combine reusable processing components for real-time applications [4]. MediaPipe Hands estimates a hand skeleton from RGB camera input using a palm detector and a hand landmark model [5]. In tutorial-style Python projects, this is commonly accessed through MediaPipe Solutions Hands, which exposes a high-level Hands object and returns landmark results through the legacy mediapipe.solutions namespace.

MediaPipe Tasks represents a newer task-oriented API. The Hand Landmarker task loads a .task model bundle and exposes explicit options for image, video, or live-stream inference. It outputs hand landmarks in image coordinates, world-coordinate landmarks, and handedness information [6]. In this project, the improved runtime treats MediaPipe Tasks as the primary backend and records fallback metadata if the Tasks backend cannot initialize. This distinction matters for reproducibility: MediaPipe Solutions describes the older tutorial-compatible API family, whereas MediaPipe Tasks describes the newer task-based backend used by the improved implementation.

The benchmark in this work is also informed by pointing-device evaluation. Fitts' law established a foundational model for target-directed movement [7], and MacKenzie later discussed its role as a research and design tool in human-computer interaction [8]. The present benchmark is not a full ISO pointing-device conformance test, but it follows the same broad principle that pointing interfaces should be evaluated through controlled target acquisition tasks rather than only through visual demonstrations [9].

3. System Design

The AI Virtual Mouse repository contains two relevant implementations. The first is a frozen tutorial-style implementation in src/video version/, which uses OpenCV [10], MediaPipe Solutions Hands, NumPy coordinate interpolation, and AutoPy to move and click the real operating-system cursor. The second is an experimental prototype under src/ai_virtual_mouse_experimental/, which separates configuration, tracking, gesture classification, cursor mapping, smoothing, benchmark logging, and report generation.

3.1 Baseline Pipeline

The baseline condition preserves the tutorial semantics. A webcam frame is processed by a hand detector, the index and middle fingertip coordinates are extracted, and finger-up states are inferred from landmark geometry. The cursor movement gesture is active when the index finger is raised and the middle finger is lowered. In this state, the index fingertip is mapped from the camera region to the output coordinate system with fixed smoothing.

Clicking is active when the index and middle fingers are both raised. If the Euclidean distance between the index fingertip and middle fingertip is below the configured threshold of 40 px, the baseline emits a click. The baseline condition does not debounce this gesture. Therefore, a held pinch can generate repeated click events across frames, which is expected to increase false clicks in a benchmark where each target requires only one final selection.

3.2 Improved Pipeline

The improved condition uses a shared hand-control pipeline. Each frame passes through a hand tracker, landmark extraction, gesture classification, cursor mapping, smoothing, debouncing, pause-state handling, and safety checks. The pipeline is designed so that the same control logic can be used by the real mouse runtime and by the benchmark runtime. It does not directly own rendering or cursor side effects; callers decide whether to apply the output to the operating-system cursor or to a simulated benchmark cursor.

The improved gesture profile used in the hand-input benchmark is the simple real-mouse profile: index-only movement, stable index-middle pinch click, and open-palm pause. Although the broader improved gesture engine contains drag and scroll classifications, drag and scroll were not evaluated in the benchmark sessions reported here.

The improved click mechanism applies debouncing to the raw pinch signal. A click is emitted only after the pinch remains active for the configured number of stable frames. The default configuration requires two stable frames, two release frames before rearming, and a 0.35 s cooldown. This design targets the main weakness of the baseline: repeated click firing while a pinch remains active.

Adaptive smoothing is used to balance stability and responsiveness. Small movements are smoothed more heavily, while larger movements use a lower smoothing factor so that the cursor can respond faster to intentional displacement. Optional calibration is represented in the configuration and mapping modules, but the saved sessions do not include participant-specific calibrated bounds. Therefore, the calibration feature should be interpreted as an available design hook rather than an experimentally isolated factor in these results.

3.3 Benchmark Safety

The benchmark mode is intentionally separated from real mouse control. It renders a target and a simulated cursor inside a Pygame window [11]. Hand-control output updates the simulated cursor position and simulated click state only. The benchmark state explicitly records that it does not control the real operating-system mouse. This design reduces risk during data collection because unstable hand input cannot move or click the user's desktop.

The real mouse runtime remains available as a separate demo mode and requires explicit permission through the runtime flag. It uses additional safety controls, including keyboard quit, open-palm pause, and a corner failsafe. These runtime features support demonstration use, but the results in this paper come from benchmark mode only.

4. Methodology

The experiment compares two conditions under the same point-and-click benchmark:

- **Baseline:** tutorial-style gesture semantics with index-only movement, index-middle pinch click, fixed smoothing, no debounce, and no pause gesture.
- **Improved:** shared hand-control pipeline with MediaPipe Tasks backend support, adaptive smoothing, stable pinch click, debounce, optional calibration hooks, and pause gesture.

Each benchmark session presents 20 targets inside a 960 x 640 window. Each target has a radius of 24 px, and the target sequence is generated from the same random seed, 20260527, to keep the task comparable across sessions. A trial is counted as a hit when the simulated cursor click falls within the current target radius. A click outside the target before the hit is counted as a false click for that trial.

The primary metrics are:

- **Hit rate:** number of targets hit divided by total target count.
- **False clicks:** total number of off-target clicks before successful target hits.
- **Total clicks:** hit clicks plus false clicks.
- **Completion time:** time from trial start to successful hit.
- **Hit-position jitter:** mean Euclidean distance between the target center and accepted hit position.

The analysis is descriptive. With only five sessions per condition and an evident outlier in the improved condition, this study does not claim statistical significance.

5. Experimental Setup

The evaluation uses ten saved hand-input benchmark sessions: five baseline sessions and five improved sessions. All sessions were collected from the repository's outputs/sessions/ directory and summarized in the generated comparison and aggregate reports.

Both conditions used hand input. During the saved benchmark sessions, metadata records MediaPipe Tasks as the actual backend used for all ten hand-input runs, including the baseline sessions. The baseline sessions are therefore best interpreted as the tutorial-style gesture condition executed through the shared benchmark environment, not as a pure empirical comparison of the older MediaPipe Solutions runtime against MediaPipe Tasks. The configured backend remains part of the condition definition, while the backend_used metadata records what actually ran.

Table 1. Point-and-click benchmark configuration.

Parameter	Value
Sessions per condition	5
Targets per session	20
Benchmark window	960 x 640 px
Target radius	24 px
Random seed	20260527
Cursor type	Simulated Pygame cursor

Parameter	Value
Operating-system mouse movement during benchmark	Disabled

6. Results and Analysis

Table 2 reports the main average results across the five sessions in each condition.

Table 2. Average performance of baseline and improved hand-input sessions.

Metric	Baseline mean	Improved mean	Interpretation
Hit rate	100.0%	100.0%	Equal task success
False clicks	192.8	4.2	Improved much lower
Total clicks	212.8	24.2	Improved much lower
Mean completion time	3.085 s	4.963 s	Baseline faster on average
Hit-position jitter	10.548 px	10.306 px	Similar, improved slightly lower

The clearest difference is click reliability. Both conditions hit every target, but the baseline generated an average of 192.8 false clicks per session. The improved condition generated an average of 4.2 false clicks per session, a reduction of approximately 97.8%. Total clicks also decreased from 212.8 to 24.2 on average, showing that the improved click mechanism emitted far fewer unintended selections.

The completion-time result is less favorable for the improved condition. Baseline sessions averaged 3.085 s per hit, while improved sessions averaged 4.963 s. Therefore, these data do not support the claim that the improved condition was faster overall. The main reason is a large outlier in the improved session 20260529T083035Z-9061f920, where the session mean reached 10.346 s. In that session, one trial took 147.854 s, while the session median completion time remained 2.104 s. The outlier substantially increased the improved condition's mean completion time.

Table 3. Session-level benchmark summary.

Session	Condition	False clicks	Total clicks	Mean completion time	Hit-position jitter
20260529T081649Z-e4795a8b	Baseline	174	194	2.449 s	10.091 px
20260529T081804Z-66b99a2e	Baseline	193	213	3.430 s	12.826 px
20260529T081926Z-d9a8080a	Baseline	214	234	3.872 s	11.263 px
20260529T082028Z-8108370 3	Baseline	185	205	2.878 s	9.323 px
20260529T082130Z-efde07f7	Baseline	198	218	2.798 s	9.234 px
20260529T082245Z-16be5208	Improved	4	24	3.324 s	9.683 px
20260529T082343Z-64ce4063	Improved	5	25	2.641 s	10.972 px
20260529T082543Z-58d86a0c	Improved	10	30	5.496 s	10.576 px
20260529T082652Z-51b683dd	Improved	2	22	3.009 s	10.559 px
20260529T083035Z-9061f920	Improved	0	20	10.346 s	9.738 px

Hit-position jitter was similar between conditions. The improved condition averaged 10.306 px compared with 10.548 px for the baseline. This difference is small and should not be treated as strong evidence of superior spatial precision. Other technical metrics from the aggregate report also show mixed behavior: improved sessions had higher mean FPS, but cursor path length and movement jitter estimates were higher. Thus, the strongest supported result is not faster or smoother movement; it is the large reduction in unintended clicks.

7. Discussion

The results indicate that the improved pipeline addressed the most visible weakness of the baseline condition. In the baseline, the click rule is directly tied to the instantaneous distance between the index and middle fingertips. When the fingertips remain close, the system can emit a click on many consecutive frames. In a benchmark where only one click is needed per target, this behavior appears as a high false-click count.

The improved pipeline changes the interaction from raw pinch detection to stable pinch click detection. The debounce state requires the pinch to be stable before firing, prevents repeated firing while the pinch remains held, and rearms only after release. This explains why total click counts in the improved condition were close to the theoretical minimum of 20 clicks per session. The improved sessions averaged 24.2 total clicks, while the baseline averaged 212.8.

The completion-time trade-off must be interpreted carefully. The improved condition was not faster on average, and the data should not be presented as showing general performance superiority. The outlier session suggests that improved click reliability can coexist with occasional slow target acquisition, possibly due to hand positioning, tracking instability, hesitation, or operator setup. The available logs identify the long trial but do not establish its cause. Therefore, the responsible interpretation is that the improved pipeline improved click reliability while preserving task success, not that it improved all usability dimensions.

The safety design is also important. By keeping benchmark interaction inside a simulated cursor environment, the experiment avoids the practical risk of accidental real desktop clicks. This makes repeated benchmarking more suitable for coursework, demonstrations, and later participant studies.

8. Limitations and Threats to Validity

This study has several limitations. First, the sample size is small: only five sessions per condition were analyzed. Second, participant diversity is not established from the available artifacts, so the results may reflect one operator's behavior, hardware, camera placement, and lighting. Third, the benchmark uses a short controlled point-and-click task and does not evaluate dragging, scrolling, text selection, menu navigation, or long-duration fatigue.

Fourth, no statistical significance test is reported. The descriptive results are sufficient to show a large false-click reduction in the collected sessions, but they are not sufficient to support broad population-level claims. Fifth, the saved hand-input sessions report MediaPipe Tasks as the actual backend used in both conditions. Consequently, the experiment does not isolate MediaPipe Solutions versus MediaPipe Tasks as an independent variable. It primarily compares baseline and improved gesture semantics within the benchmark environment.

Sixth, optional calibration is implemented as a design feature, but the reported sessions do not contain a separate calibration study. The improved condition's calibration flag should therefore not be credited as an independently measured source of improvement. Finally, the benchmark does not compare against a physical mouse. The results should not be used to claim that the virtual mouse replaces or outperforms conventional pointing devices.

9. Conclusion and Future Work

This paper compared a tutorial-style AI Virtual Mouse baseline with an improved hand-control pipeline using a controlled point-and-click benchmark. Both conditions achieved 100% hit rate across five hand-input sessions. The improved pipeline greatly reduced unintended clicks, lowering average false clicks from 192.8 to 4.2 and average total clicks from 212.8 to 24.2. Hit-position jitter remained similar. However, the improved condition was not faster on average because one improved session contained a large completion-time outlier.

The strongest supported conclusion is therefore limited and specific: the improved hand-control pipeline greatly reduced unintended clicks while preserving 100% target acquisition success in the controlled point-and-click benchmark. Future work should evaluate more participants, longer sessions, drag and scroll gestures, calibration effects, and statistically justified comparisons. A later study should also separate backend effects from gesture-pipeline effects and include a conventional physical mouse baseline only after the virtual-mouse benchmark protocol is stable.

Appendix A. Final Results and Reproducibility Artifacts

This appendix records the benchmark configuration, session identifiers, and raw session-level metrics used in the descriptive analysis. The artifacts are retained in outputs/sessions/, with one folder per benchmark session containing metadata.json, trials.csv, completion_times.svg, and report.md.

A.1 Benchmark Configuration

Table A1. Reproducibility configuration for the point-and-click benchmark.

Parameter	Value
Benchmark mode	Hand-input point-and-click benchmark
Cursor safety model	Simulated Pygame cursor; operating-system mouse movement disabled
Conditions	Baseline and improved
Sessions per condition	5
Targets per session	20
Benchmark window	960 x 640 px
Target radius	24 px
Random seed	20260527
Baseline gesture profile	Index-only movement; index-middle pinch click; no debounce
Improved gesture profile	Index-only movement; stable index-middle pinch click; open-palm pause
Baseline configured backend	mediapipe_solutions
Improved configured backend	mediapipe_tasks
Actual backend used in saved hand-input sessions	mediapipe_tasks
Debounce settings	2 stable frames, 2 release frames, 0.35 s cooldown
Primary output directory	outputs/sessions/

The actual backend field is included because the saved hand-input sessions used MediaPipe Tasks for both conditions. Therefore, these results compare baseline and improved gesture-pipeline behavior in the benchmark environment and should not be interpreted as an isolated MediaPipe Solutions versus MediaPipe Tasks backend study.

A.2 Session Inventory

Table A2. Session metadata inventory.

Session ID	Condition	Configured backend	Backend used	Gesture profile	Smoothing	Debounce	Calibration
20260529T081649Z-e4795a8b	baseline	mediapipe_solutions	mediapipe_tasks	baseline	fixed	false	false
20260529T081804Z-66b99a2e	baseline	mediapipe_solutions	mediapipe_tasks	baseline	fixed	false	false
20260529T081926Z-d9a8080a	baseline	mediapipe_solutions	mediapipe_tasks	baseline	fixed	false	false
20260529T082028Z-81083703	baseline	mediapipe_solutions	mediapipe_tasks	baseline	fixed	false	false
20260529T082130Z-efde07f7	baseline	mediapipe_solutions	mediapipe_tasks	baseline	fixed	false	false
20260529T082245Z-16be5208	improved	mediapipe_tasks	mediapipe_tasks	simple	adaptive	true	true
20260529	improved	mediapipe_	mediapipe_	simple	adaptive	true	true

Session ID	Condition	Configured backend	Backend used	Gesture profile	Smoothing	Debounce	Calibration
T082343Z-64ce4063		tasks	tasks				
20260529T082543Z-58d86a0c	improved	mediapipe_tasks	mediapipe_tasks	simple	adaptive	true	true
20260529T082652Z-51b683dd	improved	mediapipe_tasks	mediapipe_tasks	simple	adaptive	true	true
20260529T083035Z-9061f920	improved	mediapipe_tasks	mediapipe_tasks	simple	adaptive	true	true

A.3 Session-Level Metrics

Table A3. Raw session-level metrics used for analysis.

Session ID	Hit rate	False clicks	Total clicks	Mean time (s)	Median time (s)	Hit jitter (px)	Path length (px)	Mean FPS	Movement jitter (px)
20260529T081649Z-e4795a8b	100.0%	174	194	2.449	2.008	10.091	8700.54	34.4	7.139
20260529T081804Z-66b99a2e	100.0%	193	213	3.430	2.550	12.826	10674.47	33.8	7.988
20260529T081926Z-d9a8080a	100.0%	214	234	3.872	2.609	11.263	12116.95	34.5	7.280
20260529T082028Z-8108370 3	100.0%	185	205	2.878	2.209	9.323	9974.13	33.7	8.413
20260529T082130Z-efde07f7	100.0%	198	218	2.798	2.105	9.234	8851.59	33.7	7.126
20260529T082245Z-16be5208	100.0%	4	24	3.324	2.450	9.683	8811.14	33.7	8.823
20260529T082343Z-64ce4063	100.0%	5	25	2.641	2.201	10.972	8619.45	33.5	9.924
20260529T082543Z-58d86a0c	100.0%	10	30	5.496	2.256	10.576	14311.83	36.0	12.976
20260529T082652Z-51b683dd	100.0%	2	22	3.009	2.207	10.559	10180.3	36.2	11.938

Session ID	Hit rate	False clicks	Total clicks	Mean time (s)	Median time (s)	Hit jitter (px)	Path length (px)	Mean FPS	Movement jitter (px)
20260529T082652Z-51b683dd							4		
20260529T083035Z-9061f920	100.0%	0	20	10.346	2.104	9.738	13656.41	43.6	13.320

A.4 Aggregate Metrics and Outlier Note

Table A4. Aggregate means across five sessions per condition.

Metric	Baseline mean	Improved mean
Hit rate	100.0%	100.0%
False clicks	192.8	4.2
Total clicks	212.8	24.2
Mean completion time (s)	3.085	4.963
Median completion time (s)	2.296	2.244
Hit-position jitter (px)	10.548	10.306
Cursor path length (px)	10063.54	11115.83
Mean FPS	34.0	36.6
Movement jitter estimate (px)	7.589	11.396

The improved session 20260529T083035Z-9061f920 is retained in the analysis because no objective exclusion rule was defined before collection. Its mean completion time was 10.346 s due mainly to one 147.854 s trial, while its median completion time was 2.104 s. This explains why the improved condition had better click reliability but did not show faster mean completion time.

References

[1] W. Pan, C. Liu, L. Quan, X. Du, Y. Song, J. Ning, and L. Chen, "YOLO-ECN: An Efficient Tea Bud Recognition Model Based on YOLOv10s," *Smart Agricultural Technology*, vol. 14, p. 102048, 2026, doi: 10.1016/j.atech.2026.102048.

[2] D.-S. Tran, N.-H. Ho, H.-J. Yang, S.-H. Kim, and G. S. Lee, "Real-time Virtual Mouse System Using RGB-D Images and Fingertip Detection," *Multimedia Tools and Applications*, vol. 80, pp. 10473–10490, 2021, doi: 10.1007/s11042-020-10156-5.

[3] E. Uboweja, D. Tian, Q. Wang, Y.-C. Kuo, J. Zou, L. Wang, G. Sung, and M. Grundmann, "On-Device Real-Time Custom Hand Gesture Recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 4273–4277, 2023.

[4] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, *MediaPipe: A Framework for Building Perception Pipelines*, 2019, https://arxiv.org/abs/1906.08172.

[5] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, *MediaPipe Hands: On-device Real-time Hand Tracking*, 2020, https://arxiv.org/abs/2006.10214.

[6] Google AI Edge, *Hand Landmarks Detection Guide*, 2026, https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker.

[7] P. M. Fitts, "The Information Capacity of the Human Motor System in Controlling the Amplitude of Movement," *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954, doi: 10.1037/h0055392.

[8] I. S. MacKenzie, "Fitts' Law as a Research and Design Tool in Human-Computer Interaction," *Human-Computer Interaction*, vol. 7, no. 1, pp. 91–139, 1992, doi: 10.1207/s15327051hci0701_3.

[9] International Organization for Standardization, *ISO/TS 9241-411:2012 Ergonomics of Human-System Interaction – Part 411: Evaluation Methods for the Design of Physical Input Devices*, 2012, https://www.iso.org/standard/54106.html.

[10] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[11] pygame developers, *Pygame Documentation*, 2026, https://www.pygame.org/docs/.